3. Informatica IICS and PowerCenter – Interview Notes

Topics covered

Section	Topic
A	PowerCenter architecture
B	Transformations
C	Lookup transformation
D	IICS architecture and development
E	Performance and migration
F	Important interview questions

Recommended approach: Understand A and D first, then B and C, followed by E and F.

A. PowerCenter Architecture

First, understand the architecture before learning individual transformations.

1. What is Informatica PowerCenter?

Informatica PowerCenter is an ETL tool used to extract data from different sources, transform it according to business requirements, and load it into target systems.

For example:

Suppose we have employee data in an Oracle database and need to load it into a Teradata data warehouse.

```
Source – Oracle (Employee records)
        |
        v
Informatica PowerCenter (Extract → Transform → Load)
        |
        v
Target – Teradata (Data warehouse tables)
```

PowerCenter has two important services:

1. Repository Service
2. Integration Service

These are extremely important interview topics.

2. Repository Service

Simple explanation: Repository Service manages and provides access to the metadata stored in the Informatica repository.

Think of the repository as a storage area containing the ETL design information.

It stores:

- Source definitions.

- Target definitions.
- Mappings.
- Sessions.
- Workflows.
- Worklets.
- Mapping parameters and variables.
- Other reusable objects.

Example:

Suppose you create a mapping:

`m_EMPLOYEE_LOAD`

It contains:

- Oracle source.
- Expression transformation.
- Lookup transformation.
- Teradata target.

Where is the mapping definition stored?

In the PowerCenter repository.

The Repository Service manages access to this metadata.

Interview answer:

"The Repository Service manages the metadata repository. It stores ETL objects such as mappings, sessions, workflows, source and target definitions. When we develop or execute ETL workflows, the corresponding metadata is accessed through the repository."

3. Integration Service

Simple explanation: Integration Service is responsible for executing the ETL workflows and performing the actual data processing.

Think of it this way:

- Repository Service stores the instructions.
- Integration Service executes the instructions.

For example, you developed a mapping that:

1. Reads employee records.
2. Filters inactive employees.
3. Performs a lookup against a department table.
4. Loads the result into a target table.

The Integration Service executes this data-processing logic.

Interview answer:

"The Integration Service is responsible for executing workflows and sessions. It reads the mapping and workflow metadata from the repository, establishes connections to the configured sources and targets, processes the data through the transformations, and loads the results into the target system."

4. Repository and Folders

The repository stores ETL metadata, and folders help organize the objects.

For example:

```
PowerCenter Repository
│
├── Folder: EMPLOYEE_PROJECT
│   ├── Sources
│   ├── Targets
│   ├── Mappings
│   ├── Sessions
│   ├── Workflows
│   └── Worklets
│
└── Folder: CUSTOMER_PROJECT
    ├── Sources
    ├── Targets
    ├── Mappings
    └── Workflows
```

Interview question: What is the difference between a repository and a folder?

- Repository: Stores the ETL metadata.
- Folder: Organizes related ETL objects inside the repository.

5. Mapping and Sessions

These two terms are often confused.

Mapping:

A mapping defines how data moves from source to target and what transformations should happen along the way.

Example:

```
EMPLOYEE_SOURCE
       |
       v
Source Qualifier
       |
       v
Expression
       |
       v
Lookup
       |
       v
EMPLOYEE_TARGET
```

Session:

A session defines how a mapping should execute.

It contains execution-related configurations such as:

- Source and target connections.
- Commit intervals.
- Error handling.
- Session properties.

- Performance settings.
- Source and target processing options.

Easy way to remember:

Mapping = What transformation should happen?

Session = How should the mapping execute?

6. Workflow and Worklets

Workflow:

A workflow defines the execution sequence of tasks.

For example:

```
Start
  |
  v
Session 1 – Customer Load
  |
  v
Session 2 – Account Load
  |
  v
Session 3 – Transaction Load
  |
  v
End
```

A workflow can also include:

- Decision tasks.
- Command tasks.
- Email tasks.
- Timer tasks.
- Event tasks.
- Worklets.

Worklet:

A worklet is a reusable group of workflow tasks.

Imagine that five workflows need to execute the same pre-processing logic.

Instead of creating the same tasks five times, we can create one worklet and reuse it.

Interview answer:

"A workflow controls the execution of different ETL tasks, including sessions and other tasks. A worklet is a reusable collection of workflow tasks that can be called from multiple workflows."

7. Parameter Files

Parameter files allow us to externalize values instead of hardcoding them inside mappings or sessions.

Example:

```
[Global]
$$BATCH_DATE=20261104
$$SOURCE_SCHEMA=EMPLOYEE
$$TARGET_SCHEMA=DWH
```

Suppose the same workflow needs to execute in:

- Development
- Pre-production
- Production

Instead of modifying the mapping for each environment, we can configure the appropriate parameter values.

Why do we use parameter files?

1. Environment-specific configuration.
2. Reusability.
3. Easier maintenance.
4. Avoid hardcoding.
5. Simplified deployment.

Remember that parameter files and parameter scopes matter. Mapping parameters, mapping variables, session/workflow parameters and their supported scopes are not interchangeable.

8. Workflow Recovery

Workflow recovery means restarting or recovering a failed workflow from an appropriate point while maintaining data consistency.

Example:

Suppose your workflow contains three sessions:

```
Session 1 – Customer Load  SUCCESS
Session 2 – Account Load   SUCCESS
Session 3 – Transaction    FAILED
```

You don't necessarily want to execute all three sessions again.

Depending on the workflow configuration and recovery mode, you may be able to recover the failed workflow and continue from the failed task.

Important: Recovery is not automatically safe in every situation. You must consider whether previous tasks committed data, whether the failed task partially loaded data, and whether rerunning it could create duplicates.

Interview answer:

"When a workflow fails, I first identify the failed session or task and examine the session logs. I check whether the target data was partially committed. Based on the failure and workflow recovery configuration, I either recover the workflow or restart the failed session after ensuring that the target data is consistent."

B. Informatica Transformations

This is one of the most important interview sections.

Before learning individual transformations, understand what a transformation does.

A transformation is an ETL component that processes data while it moves from source to target.

Let's consider this source data:

EMPLOYEE_SOURCE

EMP_ID	EMP_NAME	DEPT_ID	SALARY
101	Rahul	10	50000
102	Arun	20	60000
103	John	10	45000
104	David	30	70000

We'll use this table throughout the examples.

1. Source Qualifier Transformation

★ Very important

What is it?

Source Qualifier is an active transformation used in PowerCenter to represent relational source data and determine how source records are read.

It can:

- Read source data.
- Filter records.
- Select specific columns.
- Perform source-side joins.
- Execute SQL overrides.

Example:

Suppose we need only employees earning more than ₹50,000.

```
SELECT *
FROM EMPLOYEE_SOURCE
WHERE SALARY > 50000;
```

The Source Qualifier can apply this filter through its source filter or SQL override configuration.

Interview answer:

"Source Qualifier is used to read relational source data in PowerCenter. We can configure source filters, SQL overrides, and source-side joins to retrieve only the required records."

2. Expression Transformation

What is it?

Expression transformation is used to perform calculations and manipulate individual records.

It is generally used for:

- String operations.

- Date calculations.
- Mathematical calculations.
- Conditional logic.
- Data type conversions.

Example:

Suppose we need to calculate annual salary.

```
ANNUAL_SALARY = SALARY * 12
```

Or create a full name:

```
FULL_NAME = FIRST_NAME || ' ' || LAST_NAME
```

Interview answer:

"Expression transformation is used for row-level calculations and data manipulation. For example, we can calculate annual salary, derive new columns, apply conditional logic, and perform string or date operations."

3. Filter Transformation

What is it?

Filter transformation allows only records satisfying a specified condition to pass through.

Example:

```
SALARY > 50000
```

Input:

EMP_ID	SALARY
101	50000
102	60000
103	45000
104	70000

Output:

EMP_ID	SALARY
102	60000
104	70000

Only records satisfying the condition pass through.

Interview answer:

"Filter transformation is used to eliminate unwanted records based on a condition. For example, if we need only active employees, we can configure the filter condition as ACTIVE_IND = 'Y'."

4. Router Transformation

What is it?

Router transformation divides incoming records into multiple output groups based on different conditions.

Example:

We need to classify employees based on salary.

```
Group 1: SALARY >= 60000
Group 2: SALARY < 60000
```

```
Employee Source
       |
       v
Router Transformation
   |                    |
   v                    v
High Salary          Low Salary
(SALARY >= 60000)    (SALARY < 60000)
   |                    |
   v                    v
Target A             Target B
```

Router also supports a default group for records that do not satisfy any configured user-defined group condition.

Interview answer:

"Router transformation is used when we need to divide records into multiple groups based on different business conditions. Unlike Filter, which generally has one output group, Router can have multiple output groups."

5. Lookup Transformation

What is it?

Lookup transformation is used to retrieve additional information from a reference table based on a matching condition.

Example:

Employee source:

EMP_ID	EMP_NAME	DEPT_ID
101	Rahul	10
102	Arun	20

Department reference table:

DEPT_ID	DEPT_NAME
10	IT
20	Finance

We need the department name in our target.

Lookup condition:

```
EMPLOYEE.DEPT_ID = DEPARTMENT.DEPT_ID
```

Output:

EMP_ID	EMP_NAME	DEPT_NAME
101	Rahul	IT
102	Arun	Finance

We will study Lookup in detail in Section C.

6. Joiner Transformation

What is it?

Joiner transformation joins data from two different pipelines.

Example:

```
Employee Source       Department Source
       |                      |
       +----------+-----------+
                  |
              Joiner
                 |
                 v
            Employee Target
```

Join condition:

```
EMPLOYEE.DEPT_ID = DEPARTMENT.DEPT_ID
```

It supports:

- Normal join.
- Master outer join.
- Detail outer join.
- Full outer join.

Important interview distinction:

Use Lookup when you need to retrieve reference information for incoming records.

Use Joiner when you need to combine two data pipelines, particularly when the sources cannot be joined directly at the database level.

7. Aggregator Transformation

What is it?

Aggregator transformation performs calculations across multiple records.

Common functions:

- SUM
- AVG
- COUNT
- MIN
- MAX

Example:

We need total salary by department.

Input:

EMP_ID	DEPT_ID	SALARY
101	10	50000
102	20	60000
103	10	45000
104	30	70000

Output:

DEPT_ID	TOTAL_SALARY
10	95000
20	60000
30	70000

We configure:

Group By: `DEPT_ID`

Expression:

```
SUM(SALARY)
```

Interview answer:

"Aggregator transformation is used to perform aggregate calculations across multiple records, such as SUM, COUNT, AVG, MIN and MAX. We can group records using group-by ports."

8. Sorter Transformation

What is it?

Sorter transformation sorts incoming records based on one or more columns.

Example:

```
ORDER BY SALARY DESC
```

Input:

EMP_ID	SALARY
101	50000
102	60000
103	45000

Output:

EMP_ID	SALARY
102	60000
101	50000
103	45000

Why do we use it?

- Sorting records.
- Preparing data for sorted operations.

- Supporting downstream processing that requires ordered input.

9. Union Transformation

What is it?

Union transformation combines records from multiple input pipelines into one output pipeline.

Example:

```
Source 1 – Current Employees
             |
             +---- Union ----> Target
             |
Source 2 – Historical Employees
```

Both pipelines must have compatible port structures and data types.

Example:

Current employees:

EMP_ID	EMP_NAME
101	Rahul

Historical employees:

EMP_ID	EMP_NAME
102	Arun

Output:

EMP_ID	EMP_NAME
101	Rahul
102	Arun

Important: Union transformation does not automatically remove duplicate records. If deduplication is required, use an appropriate downstream operation.

10. Update Strategy Transformation

★ Important for DWH interviews

What is it?

Update Strategy determines how incoming records should affect the target.

It supports four main operations:

Operation	Meaning
DD_INSERT	Insert a new record
DD_UPDATE	Update an existing record
DD_DELETE	Delete an existing record
DD_REJECT	Reject the record

Example:

Suppose we have an employee dimension table.

- Employee 101 already exists.

- Employee 102 is new.
- Employee 103 has updated salary details.

We can identify whether each record needs to be inserted or updated and apply the corresponding strategy.

Interview answer:

"Update Strategy is used to determine whether incoming records should be inserted, updated, deleted or rejected. It is particularly useful in data warehouse implementations where we handle incremental loads and existing records."

Note: The target session's update settings and target matching configuration must also support the intended operation.

11. Sequence Generator Transformation

What is it?

Sequence Generator generates sequential numeric values.

It is commonly used to generate surrogate keys in dimension tables.

Example:

EMP_NAME	SURROGATE_KEY
Rahul	1001
Arun	1002
John	1003

We can configure:

- Start value.
- Increment by.
- Current value.
- Number of cached values.

Interview answer:

"Sequence Generator is used to generate unique sequential numeric values. In data warehousing, we commonly use it to generate surrogate keys for dimension records."

Important: A generated sequence is not automatically a globally unique business key across multiple independent workflows or environments. Key management must be designed accordingly.

C. Lookup Transformation – Detailed Interview Preparation

★ High-priority interview topic

Lookup is one of the most frequently discussed Informatica transformations.

Let's understand it properly.

1. How does a Lookup transformation work internally?

Imagine we are loading employee records into a data warehouse.

We have two tables.

Employee source:

EMP_ID	EMP_NAME	DEPT_ID
101	Rahul	10
102	Arun	20
103	John	30

Department reference table:

DEPT_ID	DEPT_NAME
10	IT
20	Finance
30	HR

Our requirement is to load the employee information along with the department name.

We configure a Lookup transformation using the Department table.

Lookup condition:

```
EMPLOYEE.DEPT_ID = DEPARTMENT.DEPT_ID
```

What happens internally?

1. The incoming employee record reaches the Lookup transformation.
2. Informatica takes the employee's DEPT_ID.
3. It searches for a matching DEPT_ID in the reference table.
4. If a match is found, it returns the requested lookup columns.
5. The returned columns are passed to downstream transformations.

Output:

EMP_ID	EMP_NAME	DEPT_ID	DEPT_NAME
101	Rahul	10	IT
102	Arun	20	Finance
103	John	30	HR

Interview answer:

"Lookup transformation is used to retrieve additional information from a reference table based on specified lookup conditions. For every incoming record, Informatica searches for a matching record in the lookup source and returns the required columns. We commonly use it for retrieving surrogate keys, validating reference data and enriching incoming records."

2. Connected vs Unconnected Lookup

This is a very common interview question.

Connected Lookup:

A connected Lookup is directly connected to the mapping pipeline.

Characteristics:

- Receives input from upstream transformations.

- Can return multiple output columns.
- Can be configured as a passive or active transformation depending on the Lookup configuration.
- Can be used wherever the connected pipeline requires it.

Example:

```
Source
  |
  v
Expression
  |
  v
Connected Lookup
  |
  v
Target
```

Unconnected Lookup:

An unconnected Lookup is not directly connected to the mapping pipeline.

It is called from another transformation, usually Expression, using the :LKP function.

Example:

```
Expression
    |
    |-- Calls Lookup using :LKP
    |
Unconnected Lookup
```

Conceptually:

```
:LKP.lkp_DEPARTMENT(DEPT_ID)
```

It returns a single value through its return port.

Comparison:

Connected Lookup	Unconnected Lookup
Connected to pipeline	Not connected directly
Executes as part of pipeline	Invoked through a lookup function
Can return multiple columns	Returns one value
Generally used for normal enrichment	Useful for selective, reusable lookups
Does not require :LKP	Requires :LKP

Interview answer:

"Connected Lookup is directly connected to the mapping pipeline and can return multiple columns. Unconnected Lookup is called from another transformation using the :LKP function and returns a single value. I would use a connected Lookup for regular data enrichment and an unconnected Lookup when I need to invoke lookup logic selectively."

3. Cached vs Uncached Lookup

This is another important concept.

First understand what a cache means.

A cache is temporary memory or disk storage used by Informatica to keep lookup data available during processing.

Cached Lookup:

In a cached Lookup, Informatica loads the lookup data into its cache and uses that cache to perform subsequent searches.

Example:

Suppose our department reference table contains:

DEPT_ID	DEPT_NAME
10	IT
20	Finance
30	HR

If the lookup is cached:

1. Informatica reads the reference data.
2. Builds the lookup cache.
3. Incoming employee records search the cache.
4. Matching department information is returned.

```
Department reference table
        |
        v
Lookup Cache
  DEPT_ID: 10 -> IT
  DEPT_ID: 20 -> Finance
        ^
        |
Incoming records search the cache
```

Advantages:

- Faster repeated lookups when the cache fits the workload.
- Reduces repeated database access for lookup searches.
- Useful for reference tables that are relatively stable during execution.

Uncached Lookup:

In an uncached Lookup, Informatica does not build a full lookup cache in advance.

Instead, it accesses the lookup source to retrieve matching records as required.

Advantages:

- Avoids loading the entire reference dataset into cache.
- Useful in some scenarios where the lookup source is large or changes frequently.

Disadvantage:

- Repeated database access can increase processing time.

Comparison:

Cached	Uncached
Builds lookup cache	Does not build a full lookup cache
Searches cached data	Retrieves matches from lookup source
Good for repeated lookups	Useful for selected dynamic access patterns
Requires cache memory/disk	Can create more database overhead

4. Static vs Dynamic Lookup Cache

This distinction is particularly important in incremental ETL processing.

Static Cache:

A static cache does not change during the session's processing.

Example:

You are loading 10,000 employee records and using a department reference table.

The department reference table is read into the cache.

Even if the mapping processes thousands of records, the lookup cache remains unchanged during that execution.

Dynamic Cache:

A dynamic cache gets updated as records are processed.

This is useful when we need to check whether an incoming record already exists in the target and then update the cache as new records are inserted.

Example:

Suppose the target contains:

EMP_ID	EMP_NAME
101	Rahul

Incoming source:

EMP_ID	EMP_NAME
101	Rahul Kumar
102	Arun

Processing:

1. Lookup checks employee 101.
2. Employee 101 exists.
3. The mapping identifies it as an update.
4. Employee 102 does not exist.
5. It is identified as an insert.
6. The dynamic cache is updated as applicable.

Comparison:

Static Cache	Dynamic Cache
Does not change during processing	Changes during processing
Useful for reference lookups	Useful for incremental loads
Read-only lookup behaviour	Can reflect processed inserts and updates

Interview answer:

"Static cache remains unchanged during session execution. Dynamic cache gets updated as records are processed. I would consider dynamic lookup caching when implementing incremental loads where we need to identify existing and new records and maintain the lookup state during the load."

5. Persistent Lookup Cache

A persistent cache is a lookup cache saved beyond the current session execution so that it can potentially be reused in subsequent executions.

Example:

Suppose we have a department reference table containing 5 lakh records.

Building the cache every time may add overhead.

If the reference data does not change frequently, a persistent cache can be useful.

Instead of rebuilding the cache every time, Informatica can reuse an existing compatible cache.

Important considerations:

- The lookup definition and cache compatibility must be maintained.
- If reference data changes, the cache must be refreshed or rebuilt.
- A stale cache can return outdated information.

Interview answer:

"Persistent Lookup Cache allows Informatica to retain lookup cache data between session executions. It can reduce cache-building overhead when lookup data is relatively stable. However, we need to manage cache refreshes to avoid stale reference data."

6. What happens when a Lookup returns multiple matching records?

Consider this reference table:

DEPT_ID	DEPT_NAME
10	IT
10	Technology
20	Finance

Our incoming record has:

```
DEPT_ID = 10
```

There are two matching records.

What happens?

The behaviour depends on the Lookup configuration and implementation.

In PowerCenter, for multiple matching rows, you can configure the Lookup policy to return:

- The first matching value.
- The last matching value.
- Any matching value.
- An error, depending on the multiple-match policy.

For deterministic results, use an appropriate ordering and lookup design rather than relying on an arbitrary first or last match.

Interview answer:

"If multiple matching records exist, the result depends on the configured multiple-match policy. To avoid unpredictable results, I would first investigate why the lookup key is not unique and, if necessary, apply appropriate filtering or aggregation to ensure a single deterministic match."

7. What are Lookup Conditions?

Lookup conditions determine how incoming records are matched against the lookup source.

Example:

```
SOURCE.EMP_ID = LOOKUP.EMP_ID
```

We can also have multiple conditions:

```
SOURCE.EMP_ID = LOOKUP.EMP_ID
AND SOURCE.COUNTRY_CODE = LOOKUP.COUNTRY_CODE
```

The lookup returns matching data based on these conditions.

Important: Matching conditions and the returned lookup columns are different things.

- Lookup condition: Determines which row to find.
- Return port: Determines which column value to return.

8. Lookup Performance Tuning

Suppose your mapping is processing 50 million records and taking 4 hours.

A Lookup transformation could be contributing to the delay.

How can you optimize it?

1. Select only the required lookup columns.
2. Use appropriate lookup conditions.
3. Create indexes on lookup condition columns where applicable.
4. Consider cached Lookup for frequently accessed reference data.
5. Use persistent caching when reference data is stable.
6. Avoid unnecessarily large lookup datasets.
7. Consider database-side joins when both datasets are in the same database.
8. Monitor cache size, memory and disk usage.
9. Avoid unnecessary repeated lookups.
10. Review the execution logs and actual bottleneck before changing configurations.

Interview answer:

"For Lookup performance tuning, I first examine the lookup source size, lookup conditions and cache behaviour. I try to retrieve only the required columns, ensure appropriate indexing on lookup keys, and use caching when it is beneficial. If both datasets reside in the same database, I also evaluate whether a database-side join or pushdown processing would be more efficient."

D. IICS Architecture and Development

Now let's move to IICS.

1. What is IICS?

IICS stands for Informatica Intelligent Cloud Services. Its current cloud data management platform is part of Informatica's Intelligent Data Management Cloud (IDMC).

It is a cloud-based integration platform that enables data integration across different systems.

For example:

- Teradata to BigQuery.
- Oracle to BigQuery.
- SQL Server to GCS.
- Cloud applications to data warehouses.

The major difference from traditional PowerCenter is that IICS provides a cloud-managed platform with execution through configured runtime environments, including Secure Agents.

2. IICS Architecture

Let's understand the architecture.

```
IICS / IDMC Cloud
(Design, configuration, scheduling, monitoring and metadata management)
              |
              v
Secure Agent
(Executes integration tasks in the configured runtime environment)
        |                          |
        v                          v
Source                         Target
Teradata / Oracle              BigQuery / GCS
```

The cloud platform manages design and orchestration capabilities, while the Secure Agent performs the runtime work for supported integration tasks.

3. What is a Secure Agent?

★ Very important

Simple explanation:

Secure Agent is a lightweight runtime application installed on a machine or server that enables IICS to communicate with configured data sources and targets and execute integration tasks.

Think of it as the execution engine between the cloud platform and your enterprise systems.

Example:

Your source database is inside your company's network, but IICS is hosted in the cloud.

How can IICS access the source database?

A Secure Agent installed inside the company's network can connect to the database using the configured connection and network access.

It communicates with the IICS cloud platform and executes assigned tasks.

Interview answer:

"Secure Agent is a runtime component installed on a machine within the network. It enables IICS to connect to enterprise databases and cloud services and execute integration tasks. It acts as the bridge between the IICS cloud platform and the configured source and target systems."

4. Runtime Environments

A runtime environment defines the execution environment used by IICS tasks.

For example:

- Development runtime.
- Pre-production runtime.
- Production runtime.

These may use separate Secure Agents or Secure Agent groups.

Example:

Environment	Runtime
DEV	DEV_AGENT
PRE-PROD	PP_AGENT
PROD	PROD_AGENT

When configuring a Mapping Task, we select the appropriate runtime environment.

Interview answer:

"Runtime Environment specifies where an IICS task should execute. We configure the appropriate Secure Agent or agent group for each environment to maintain separation between development, pre-production and production processing."

5. Connections

A connection contains the configuration required to communicate with a source or target system.

Examples:

- Teradata connection.
- BigQuery connection.
- Oracle connection.
- GCS connection.

A connection typically includes:

- Connection type.
- Host or endpoint.
- Database or project information.
- Authentication details.

- Additional connector-specific settings.

Example:

Suppose we need to connect to BigQuery.

The connection would contain the necessary GCP project, authentication and connector configuration.

The Mapping Task can then use that connection to access BigQuery.

Interview answer:

"Connections store the configuration required for IICS to access different source and target systems. We create reusable connections and assign them to mappings or tasks depending on the integration requirement."

6. Mappings

A Mapping defines the data flow and transformation logic.

Example:

```
Teradata Source
      |
      v
Expression
      |
      v
Lookup
      |
      v
Filter
      |
      v
BigQuery Target
```

It defines:

- Source objects.
- Target objects.
- Field mappings.
- Transformation logic.
- Data flow.

Remember:

A mapping defines what should happen to the data, not the complete execution schedule.

7. Mapping Tasks

A Mapping Task is the executable object created using a mapping.

Think of it this way:

- Mapping = Design.
- Mapping Task = Execution configuration.

A Mapping Task includes configurations such as:

- Mapping selection.
- Runtime environment.

- Connections.
- Parameter values.
- Source and target overrides.
- Scheduling and execution settings, where applicable.

Example:

```
Mapping: m_EMPLOYEE_LOAD

Mapping Task: mt_EMPLOYEE_LOAD_DEV

Mapping Task: mt_EMPLOYEE_LOAD_PROD
```

The same mapping can potentially be reused with different task configurations.

8. Taskflows

★ Very important

A Taskflow orchestrates multiple IICS tasks and controls their execution sequence.

Example:

```
Start
  |
  v
Customer Load
  |
  v
Account Load
  |
  v
Transaction Load
  |
  v
End
```

Taskflows can support:

- Sequential execution.
- Parallel execution.
- Decision logic.
- Conditional paths.
- Error handling.
- Reusable task orchestration patterns.

Practical example:

Suppose you have three Mapping Tasks.

1. Load customer data.
2. Load account data.
3. Load transaction data.

You want the account load to execute only after customer load succeeds.

You can orchestrate this through a Taskflow.

Interview answer:

"Taskflow is used to orchestrate multiple IICS tasks. It controls execution sequence, dependencies, conditional processing, parallel execution and error handling. For example, we can execute a customer load first, followed by account and transaction loads based on successful completion."

9. Parameterization in IICS

Parameterization allows us to pass dynamic values into mappings and tasks instead of hardcoding them.

Example:

Suppose we have a mapping that reads data for a particular business date.

Instead of hardcoding:

```
WHERE BUSINESS_DATE = '2026-11-04'
```

We can use a parameter:

```
WHERE BUSINESS_DATE = $$BUSINESS_DATE
```

Then provide the value through the applicable parameter configuration.

Benefits:

- Reusability.
- Environment-specific execution.
- Dynamic date processing.
- Easier maintenance.
- Reduced hardcoding.

10. Parameter Files in IICS

IICS supports parameter files for applicable mapping/task execution configurations.

Example:

```
[Global]
$$BUSINESS_DATE=20261104
$$SOURCE_TABLE=EMPLOYEE_SOURCE
$$TARGET_TABLE=EMPLOYEE_TARGET
```

This allows the same mapping logic to run with different values.

For example:

```
DEV       → DEV_SOURCE / DEV_TARGET
PRE-PROD  → PP_SOURCE / PP_TARGET
PROD      → PROD_SOURCE / PROD_TARGET
```

Important interview distinction:

Do not assume that PowerCenter and IICS use identical parameter file syntax or execution behaviour. IICS parameter support depends on the task type, mapping configuration, parameter scope and runtime setup.

11. Workflow Monitoring

IICS provides monitoring capabilities to track integration task execution.

You can monitor:

- Task status.
- Start and end time.
- Execution duration.
- Processed rows.
- Failed rows.
- Error messages.
- Runtime information.

Common execution statuses include:

- Success.
- Failed.
- Running.
- Suspended or other task-specific states.

Practical example:

Suppose a BigQuery load fails.

Your troubleshooting process:

1. Open the IICS monitoring page.
2. Identify the failed Mapping Task.
3. Open the execution details.
4. Review the error message.
5. Examine the Secure Agent logs if necessary.
6. Identify whether the problem is source connectivity, SQL, transformation logic, permissions or target loading.
7. Fix the issue and rerun after checking target data consistency.

12. Error Handling

Error handling means identifying, capturing and managing failures during ETL execution.

There are several types of errors.

Error type	Example
Connection error	Source database unavailable
Data error	Invalid date format
Transformation error	Invalid expression
Target error	Missing column or permission
Runtime error	Secure Agent unavailable
Business validation error	Missing mandatory business key

A good ETL implementation should:

- Capture execution errors.
- Log relevant details.

- Track failed records where applicable.
- Avoid uncontrolled duplicate loads.
- Send failure notifications.
- Support safe recovery or restart.

13. Deployment and Migration

Deployment means moving developed ETL objects from one environment to another.

Example:

```
DEV → PRE-PROD → PROD
```

A typical deployment process:

1. Develop the mapping.
2. Perform unit testing.
3. Validate source and target connections.
4. Validate parameter configurations.
5. Deploy the required assets.
6. Configure environment-specific connections and runtime.
7. Perform integration testing.
8. Obtain approval.
9. Deploy to production.
10. Monitor the first production execution.

Interview answer:

"We follow a controlled deployment process. We develop and unit-test the mappings in Development, validate the changes in Pre-production, and then promote the required assets to Production. We ensure that connections, runtime environments, parameters and task dependencies are correctly configured before production execution."

E. Performance and Migration

This section is especially important because interviewers often ask scenario-based questions rather than definitions.

1. What is Pushdown Optimization?

★ Frequently asked

Let's understand this with a practical example.

Suppose we have:

- Source: Teradata.
- Target: BigQuery.
- Requirement: Filter employees whose salary is greater than 50,000.

There are two ways to process this data.

Approach 1: Without Pushdown Optimization

The ETL engine reads all employee records from Teradata, brings them into the integration runtime, and applies the filter.

```
Teradata
   |
   | All records
   v
IICS Runtime
   |
   | Apply SALARY > 50000
   v
Filtered records
   |
   v
BigQuery
```

If the source contains 10 million records but only 1 million satisfy the condition, we are transferring unnecessary records.

Approach 2: With Source-side Pushdown

When supported and configured, IICS translates eligible transformation logic into SQL and executes it at the source.

```
SELECT *
FROM EMPLOYEE_SOURCE
WHERE SALARY > 50000;
```

Now only the required records are transferred.

Comparison:

Approach	Records read by ETL / transferred
Without pushdown	10M records read by ETL
With source filter	1M records transferred

Illustrative example: assumes only 10% of source records satisfy the filter.

Types of Pushdown Optimization:

- Source-side pushdown: Executes eligible transformation logic at the source.
- Target-side pushdown: Executes eligible processing at the target.
- Full pushdown: Executes all eligible logic in the database.
- Partial pushdown: Executes some logic in the database and the remaining logic in the integration runtime.

Important: Pushdown is connector-, transformation- and configuration-dependent. Not every transformation can be pushed down, and cross-database processing may not be eligible for full pushdown.

Interview answer:

"Pushdown Optimization is a performance optimization technique where Informatica converts eligible transformation logic into database SQL and executes it within the database instead of processing everything in the integration engine. This reduces unnecessary data movement and can improve performance."

2. Source-side vs Target-side Processing

Let's understand the difference.

Source-side processing:

Processing takes place at the source database.

Example:

```
SELECT *
FROM EMPLOYEE
WHERE ACTIVE_IND = 'Y';
```

The source database filters the records before sending them to IICS.

Target-side processing:

Processing takes place at the target database.

For example, when eligible operations are pushed down to the target, the target database can execute SQL to process or transform data there.

Comparison:

Source-side	Target-side
Processing at source	Processing at target
Reduces unnecessary source data transfer	Uses target database processing capabilities
Useful for source filtering and aggregation	Useful for eligible target-side processing

3. Lookup Caching Optimization

We discussed Lookup caching earlier.

For interviews, remember these points:

- Use cached Lookup for repeated reference lookups when beneficial.
- Use persistent cache when reference data is stable and cache reuse is appropriate.
- Use dynamic cache when lookup state needs to change during processing.
- Reduce unnecessary lookup columns.
- Ensure efficient lookup conditions.
- Evaluate database indexes and cache size.

Scenario:

Your mapping takes 3 hours to process 10 million records. The Lookup transformation takes most of the execution time.

What would you do?

Answer:

"First, I would review the Lookup source size, lookup conditions and cache configuration. I would check whether the lookup is repeatedly accessing the database, whether caching is suitable, and whether the lookup source can be reduced. I would also consider performing the join at the database level if both datasets reside in the same database."

4. Session Performance

What if a PowerCenter session is running slowly?

We should not immediately assume the mapping logic is responsible.

We need to identify the actual bottleneck.

Some common causes:

Area	Possible issue
Source	Slow SQL query
Network	High data transfer latency
Lookup	Large lookup dataset or poor caching
Aggregator	Large grouping workload
Sorter	High sorting overhead
Target	Slow inserts or updates
Database	Resource contention
Runtime	Insufficient CPU or memory

How would you troubleshoot?

1. Check session logs.
2. Identify which transformation or processing stage consumes time.
3. Review source query execution.
4. Check lookup cache statistics.
5. Review target loading performance.
6. Check database and runtime resource utilization.
7. Evaluate partitioning or parallelism where supported.
8. Apply the required optimization.
9. Compare execution time before and after the change.

Interview answer:

"When troubleshooting a slow session, I first identify the bottleneck using session logs and performance statistics. I investigate source query performance, lookup caching, transformation processing, network overhead and target loading. Based on the findings, I optimize the specific bottleneck rather than making random configuration changes."

5. PowerCenter to IICS Migration Strategy

★ Very important for your experience

Suppose an organization has 500 PowerCenter workflows and wants to migrate them to IICS.

Would you simply recreate all workflows manually?

Not necessarily.

We follow a structured migration approach.

Step 1: Assessment

Understand the existing PowerCenter implementation.

Identify:

- Mappings.
- Sessions.
- Workflows.

- Worklets.
- Parameter files.
- Source and target connections.
- Custom SQL.
- Reusable transformations.
- Dependencies.
- Scheduling configurations.

Step 2: Compatibility Analysis

Check which PowerCenter objects and transformations are supported in IICS.

Identify:

- Unsupported transformations.
- Custom SQL dependencies.
- Database-specific functions.
- Parameter differences.
- Workflow logic differences.
- Connection differences.

Step 3: Migration or Conversion

Use appropriate Informatica migration/conversion capabilities where supported.

Convert or recreate:

- Mappings.
- Mapping Tasks.
- Taskflows.
- Connections.
- Parameters.

Not every PowerCenter workflow is automatically convertible without modification.

Step 4: Configuration

Configure:

- Secure Agents.
- Runtime environments.
- Source connections.
- Target connections.
- Parameter values.
- Environment-specific configurations.

Step 5: Validation

This is one of the most important steps.

Suppose we are migrating an existing PowerCenter workflow that loads 10 million records into a BigQuery target.

We execute the old and new implementations against equivalent input data and compare the results.

Validation checklist:

Validation	What to check
Source count	Records extracted
Target count	Records loaded
Data quality	Nulls and invalid records
Business logic	Transformation results
Lookup	Correct matching records
Aggregation	Correct totals
Duplicate check	Duplicate business keys
Reconciliation	Source and target differences
Performance	Execution time

We can also use SQL:

```
SELECT COUNT(*)
FROM TARGET_TABLE;
```

And perform data-level comparison using:

```
SELECT * FROM OLD_TARGET
EXCEPT DISTINCT
SELECT * FROM NEW_TARGET;
```

Run the reverse comparison too:

```
SELECT * FROM NEW_TARGET
EXCEPT DISTINCT
SELECT * FROM OLD_TARGET;
```

If both return no rows, the compared datasets match for the selected columns and comparison conditions.

Step 6: Deployment

After successful testing:

```
Development
    |
    v
Pre-production
    |
    v
Production
```

Ensure:

- Production connections are configured.
- Parameters are correct.
- Scheduling dependencies are migrated.
- Taskflows execute correctly.
- Monitoring and alerting are configured.
- Rollback or restart procedures are ready.

Interview answer:

"We followed a structured PowerCenter-to-IICS migration approach. First, we assessed the existing mappings, workflows, reusable components, parameter files and dependencies. Then we evaluated transformation compatibility and migrated or recreated the required objects in IICS. We configured Secure Agents, connections and runtime environments, followed by functional and data-level validation. We compared source and target counts, transformation results, duplicate records and execution performance before promoting the workflows to Production."

6. Workflow Conversion Validation

An interviewer may ask:

"How do you ensure that a migrated workflow behaves exactly like the original PowerCenter workflow?"

The answer is not simply that the new workflow runs successfully.

You need to validate functional equivalence.

Example:

Original PowerCenter:

```
WF_EMPLOYEE_LOAD
   |
   v
Session 1
   |
   v
Session 2
   |
   v
Session 3
```

Migrated IICS:

```
TF_EMPLOYEE_LOAD
   |
   v
Mapping Task 1
   |
   v
Mapping Task 2
   |
   v
Mapping Task 3
```

We check:

1. Same execution sequence.
2. Same source and target objects.
3. Same transformation logic.
4. Same lookup conditions.
5. Same filter conditions.
6. Same parameter values.
7. Same error handling.
8. Same dependency handling.
9. Same final target data.

10 Acceptable performance.

Important: We must validate the actual business results, not just the successful conversion of the ETL objects.

7. Production Failure Troubleshooting

Imagine your IICS production Mapping Task fails.

How would you handle it?

Follow this sequence:

1. **Identify the failure** – Check task status and identify the failed Mapping Task or Taskflow step.
2. **Check execution logs** – Review error messages, row counts, runtime details and execution logs.
3. **Identify the root cause** – Check connectivity, credentials, SQL, source data, transformation logic, target schema and permissions.
4. **Check data consistency** – Verify whether partial data was committed into the target.
5. **Fix the issue** – Apply the required code, configuration or infrastructure correction.
6. **Rerun safely** – Ensure duplicate or incomplete records are handled before restarting.
7. **Validate and communicate** – Confirm target counts, data quality and execution status, then update stakeholders.

Interview answer:

"When a production workflow fails, I first check the IICS monitoring details and identify the failed task. I examine the execution logs to determine whether the issue is related to connectivity, source data, transformation logic, permissions or target loading. Before rerunning, I check whether any partial records were committed and ensure the restart will not create duplicate data. After resolving the issue, I rerun the task, validate the target data and communicate the resolution."

F. Important Interview Questions and Answers

Now let's consolidate the most frequently asked questions.

Q1. How does a Lookup transformation work internally?

Answer:

"Lookup transformation receives an incoming record and searches a reference dataset using configured lookup conditions. Depending on the cache configuration, Informatica either searches cached lookup data or accesses the lookup source. Once a match is found, the required lookup columns are returned to the downstream transformation."

Q2. When would you use Joiner instead of Lookup?

Answer:

"I use Lookup when I need to retrieve additional information from a reference table based on incoming records. I use Joiner when I need to combine two independent data pipelines, particularly when the sources cannot be joined directly at the database level."

Q3. What is Pushdown Optimization?

Answer:

"Pushdown Optimization allows eligible transformation logic to be converted into SQL and executed within the database. Instead of transferring all data to the Informatica engine for processing, we can leverage the database's processing capabilities. This can reduce data movement and improve performance."

Q4. How do you troubleshoot a slow mapping?

Answer:

"I start by checking the execution logs and identifying where most of the processing time is consumed. I investigate source query performance, lookup caching, transformation processing, network overhead and target loading. Once the bottleneck is identified, I apply the relevant optimization and compare execution performance."

Q5. How do you migrate a PowerCenter workflow to IICS?

Answer:

"I first assess the existing PowerCenter mappings, sessions, workflows, worklets, parameters and dependencies. I then evaluate compatibility, migrate or recreate the required objects, configure connections and Secure Agents, and validate the converted workflow. Finally, I compare data counts, transformation results and execution performance before production deployment."

Q6. How do you recover a failed workflow?

Answer:

"I identify the failed task and examine the execution logs. I check whether earlier tasks completed successfully and whether any partial data was committed. Depending on the failure and recovery configuration, I either recover the workflow or restart the failed task after ensuring target data consistency."

Q7. What happens when a Taskflow fails in the middle?

Answer:

"When a Taskflow fails in the middle, the failed step and subsequent execution depend on the configured fault-handling and execution logic. Previously completed tasks may already have committed their data. I identify the failed step, review its error details, verify the status of previous loads, resolve the issue and restart or rerun the appropriate portion of the Taskflow while preventing duplicate processing."

G. Quick Revision – Important Differences

These comparisons are useful for rapid interview revision.

1. PowerCenter vs IICS

PowerCenter	IICS
Traditional on-premises ETL platform	Cloud-based integration platform
Repository Service	Cloud-managed metadata and asset services
Integration Service	Secure Agent/runtime execution
Mapping	Mapping
Session	Mapping Task execution configuration
Workflow	Taskflow
Workflow Monitor	IICS Monitor
Repository folders	Projects and folders
Parameter files	Parameterization and supported parameter files

These are conceptual equivalents, not exact one-to-one replacements.

2. Important Transformation Differences

Transformation	Main purpose
Source Qualifier	Read relational source data
Expression	Row-level calculations
Filter	Remove unwanted records
Router	Split records into multiple groups
Lookup	Retrieve reference information
Joiner	Join two pipelines
Aggregator	Perform aggregate calculations
Sorter	Sort records
Union	Combine multiple pipelines
Update Strategy	Insert/update/delete/reject
Sequence Generator	Generate sequential values

3. Lookup Revision

Concept	Remember this
Connected	Directly connected to pipeline
Unconnected	Called using :LKP
Cached	Builds lookup cache
Uncached	Does not build a full cache
Static	Cache remains unchanged
Dynamic	Cache gets updated
Persistent	Cache retained between executions
Multiple matches	Depends on configured policy

4. Five scenarios you should be able to answer

These are particularly useful for someone working in a real ETL development environment.

Scenario 1: Lookup performance

Your mapping is taking too long because the lookup table contains millions of records.

How would you optimize it?

Expected topics: lookup conditions, cache, indexes, filtering lookup data, database-side joins.

Scenario 2: Duplicate records

Your target table contains duplicate employee records after rerunning a failed workflow.

How would you prevent this?

Expected topics: primary/business keys, deduplication, incremental load logic, MERGE, restartability and idempotency.

Scenario 3: Migration mismatch

Your PowerCenter workflow loads 10 million records, but the IICS version loads only 9.8 million.

How would you investigate?

Expected topics: source filters, lookup conditions, rejected records, datatype conversions, null handling, parameters and target counts.

Scenario 4: Production failure

Your Taskflow fails after the first two Mapping Tasks complete successfully.

Would you rerun the entire Taskflow?

Expected topics: execution status, committed data, dependencies, restartability and duplicate prevention.

Scenario 5: Slow ETL

Your mapping takes four hours in Production but only one hour in Pre-production.

What would you investigate?

Expected topics: data volume, runtime resources, database workload, query plans, agent performance, network, partitioning and concurrency.

H. My recommended study priority

You don't need to give equal preparation time to every topic.

For your ETL Developer / Data Engineer interviews, I recommend this order:

Priority	Topics
High	Lookup, Connected vs Unconnected
High	Cached vs Uncached, Static vs Dynamic
High	Secure Agent, Mapping Task, Taskflow
High	PowerCenter vs IICS migration
Medium	Pushdown Optimization
Medium	Performance tuning
Medium	Workflow recovery and error handling
Foundation	Remaining transformations and architecture